

Replication Freshness Is Consumer-Relative and Witnessed: Sealed Measurement of Staleness-Monitor Vacuity Across Database Substrates

Andrew H. Bond, *Senior Member, IEEE*

Abstract

Replicated data stores expose *staleness monitors*: a replica-lag threshold that certifies a replica “fresh enough” to read. We show, and measure, that such monitors are *consumer-relative vacuous*: no single lag threshold serves heterogeneous consumers, because staleness is a property of *what a given consumer reads*, not of the replica in aggregate. We formalize a *footprint certificate*—the divergence the actual consumer feels, $\text{tr}(P_C \Sigma)$ for a consumer read-operator P_C and a replica drift Σ —graded against an independent *witness*: the write-ahead-log sequence number, the oplog timestamp, or the replica’s own replay position. Under a sealed-preregistration protocol (bars committed before graded runs, a coded cooling-off, manipulation checks, disjoint seeds), we measure the false-clear rate of the naive lag monitor and the footprint certificate across four substrates: a PostgreSQL lab, a MongoDB replica set, a production PostgreSQL instance with footprints derived from live query statistics, and a geo-distributed PostgreSQL fleet spanning three real datacenters. In every cell the naive monitor false-clears near 0.5 (two-sided: it both admits stale reads and rejects fresh ones), while the witnessed footprint certificate holds at 0.01–0.06; on the geo-fleet a *nearest-certified* router is both fresher than nearest-replica routing and more local than least-lag routing. The result is the database instance of a program thesis: reliability is a property of the consumer relation, and a monitor that does not read the consumer’s footprint, graded against a witness, is measurably vacuous.

Index Terms

Replication, consistency, staleness, read routing, write-ahead log, PostgreSQL, MongoDB, pre-registration, CAP, PACELC.

I. INTRODUCTION

A read served from a replica may be stale. Systems therefore ship *staleness monitors*: bounded-staleness reads, maximum replica lag thresholds, “read-your-writes” flags. Each is a *certificate*—an assertion that a replica is fresh enough to serve a read. This paper asks what such certificates actually certify, and measures how often they are wrong.

The central finding is that a replica-lag threshold is the wrong object. Staleness is not a scalar property of a replica; it is a property of the relation between a replica’s divergence and *what a particular consumer reads*. A consumer that reads only slowly-changing objects tolerates a lag that would corrupt a consumer reading a hot object. Consequently no single lag threshold serves a heterogeneous workload: set it low and the monitor false-alarms on consumers who do not care; set it high and it admits stale reads to consumers who do. We measure this two-sided failure directly: a hot consumer sees a truth-staleness rate near 0.5 above the threshold, while a cold consumer pays a false-alarm rate near 0.9 below it.

The repair is to certify the quantity the consumer feels. Let P_C be a consumer’s *footprint*—a positive operator encoding which objects (or query results) it reads and how heavily—and let Σ be the replica’s drift (the divergence of replicated state from the primary). The consumer’s felt staleness is $\text{tr}(P_C \Sigma)$; a *footprint certificate* clears a read iff this quantity is within tolerance. Crucially, Σ is witnessed natively: the write-ahead log (WAL) sequence number in PostgreSQL, the oplog timestamp in MongoDB, and the replica’s replay position are exactly the independent ground truth of how far behind the replica is, per object. The certificate is thus *consumer-relative* (it reads P_C) and *witnessed* (it grades against the log), the same $\text{tr}(P_C \Sigma)$ form we develop generally in the Observation-Theory program [1] and ship publicly for compression [2].

A. Contributions

- 1) The footprint certificate for replication freshness (Section II): $\text{tr}(P_C \Sigma)$ witnessed by the replication log, and the two-sided vacuity of scalar lag thresholds.
- 2) Four sealed cells (Section IV) on unmodified substrates: a PostgreSQL lab, a MongoDB replica set (a second substrate, a second witness), a *production* PostgreSQL instance whose footprints are derived from live query statistics, and a geo-distributed fleet across three real datacenters. All pass every bar and manipulation check on disjoint graded seeds.

Manuscript prepared August 2026. The author is with the Department of Computer Engineering, San José State University, San José, CA 95192 USA (e-mail: andrew.bond@sjsu.edu; ORCID 0009-0003-2599-6158).

Every empirical claim is governed by a sealed pre-registration and a coded verdict runner committed before the graded runs, archived with the program. Substrates are unmodified PostgreSQL and MongoDB and a geo-distributed PostgreSQL fleet on the National Research Platform.

TABLE I

SEALED DATABASE CELLS, GRADED ON DISJOINT SEEDS. “NAIVE” IS THE SCALAR REPLICA-LAG MONITOR (MIN–MAX OVER GRADED SEEDS; NEAR 0.5 = TWO-SIDED VACUITY); “WITNESSED” IS THE FOOTPRINT CERTIFICATE GRADED AGAINST THE REPLICATION LOG. EVERY BAR AND MANIPULATION CHECK PASSED ON EVERY SEED.

Cell	Substrate	Witness	naive FC	witnessed FC	verdict
XPROTO-PG	PostgreSQL (lab)	WAL LSN	0.50	0.05–0.06	PASS
XPROTO-MG	MongoDB replica set	oplog timestamp	0.45–0.48	0.03–0.05	PASS
XPROTO-PGX	PostgreSQL (production)	WAL LSN, derived footprint	0.46–0.47	0.01–0.03	PASS
XPROTO-GEO	PostgreSQL fleet (3 datacenters)	WAL LSN per replica	nearest-certified dominates (§IV-C)		PASS

- 3) A measurement, on real WAN geography, that a nearest-certified read router dominates both nearest-replica and least-lag routing on the axes each optimizes (Section IV-C).
- 4) A placement of the result in the consistency design space (Section V) and the CAP/PACELC tradeoff (Section VI): the certificate is the constructive form of the staleness face of consistency.

We credit the deployed mechanisms—probabilistically bounded staleness [3], commit-wait [4], session guarantees [5]—and measure what a consumer-relative, witnessed, calibrated certificate adds over each.

II. FOOTPRINT CERTIFICATES AND THEIR WITNESSES

Definition 1 (Footprint certificate). *For a consumer with read-operator $P_C \succeq 0$ (which objects/queries it reads, weighted) and a replica with per-object drift Σ , the felt staleness is $d_C = \text{tr}(P_C \Sigma)$; the certificate clears a read from that replica iff $d_C \leq \epsilon$ for a tolerance ϵ .*

Definition 2 (Witness). *An independent measurement of Σ from the replication substrate itself: the WAL log-sequence number (LSN) reached by the replica versus the primary in PostgreSQL, the oplog applied-timestamp in MongoDB, or the replica’s replay position. The witness is not an inference; it is the ground truth the substrate already maintains.*

Definition 3 (False-clear rate). *FC is the rate at which a monitor clears a read whose witnessed felt staleness exceeds tolerance (admitting stale) or rejects a read whose felt staleness is within tolerance (false alarm). A scalar lag monitor incurs both; a footprint certificate reads P_C and grades against the witness, and incurs neither beyond sampling.*

Proposition 1 (Two-sided vacuity of scalar lag, informal). *For a workload with heterogeneous footprints there is no replica-lag threshold that simultaneously bounds admitted-stale rate for hot consumers and false-alarm rate for cold consumers; the two error rates trade off along the threshold and meet near $\frac{1}{2}$ at the crossover, independent of the threshold.*

This is the object we measure. The consumer-relativity is not a modeling choice; it is forced by the workload, and the witness makes the felt staleness observable per read.

III. SEALED-PREREGISTRATION MEASUREMENT

Each cell commits, before any graded run, three bars—B1 (the naive lag monitor is vacuous, false-clear $\geq$ a threshold near $\frac{1}{2}$), B2 (the footprint certificate holds, false-clear $\leq$ a small tolerance), B3 (dominance)—and manipulation checks that are also bars: that the induced staleness is real, that a fresh replica is certified sane, that the certificate is non-degenerate, and, for the production cell, that the footprint derivation from live statistics is exact. A verdict runner refuses to grade unless the pre-registration was sealed at least one day after construction (enforced in code), and grades on seeds disjoint from the shakedown used to calibrate the bars. Failing runs are kept as executed. This is the same protocol used for the radio-access instance and the routing instance of the program.

IV. FOUR SEALED CELLS

Table I summarizes the sealed, graded results.

A. PostgreSQL lab and the MongoDB twin

The base cell runs a PostgreSQL primary and a replica with a bounded apply delay; a seeded write schedule drives heterogeneous consumer footprints. The naive monitor—a single replica-lag threshold—is two-sidedly vacuous: below the apply delay a cold consumer pays a 0.90 false-alarm rate (an idle replay-timestamp pathology inflates apparent lag), while above it a hot consumer’s admitted-stale rate is 0.50; the min–max false-clear across graded seeds sits at 0.50. The WAL-witnessed footprint certificate— $\text{tr}(P_C \Sigma)$ against the replica’s replay LSN—holds at 0.05–0.06. The MongoDB cell reproduces the result on a second substrate and a *second witness*: a replica set with a delayed secondary, the oplog applied-timestamp

as Σ . Graded, the naive monitor is 0.45–0.48 and the oplog-witnessed certificate 0.03–0.05. Two substrate-specific pitfalls, found and fixed during shakedown, are reported in the cell: a majority write-concern that erases the modeled staleness, and reading a secondary’s applied time through the primary’s heartbeat (which itself adds staleness); the certificate must read the secondary’s own member position.

B. Production PostgreSQL with derived footprints

The production cell earns two graduations over the lab. First, the consumer footprints P_C are not seeded but *derived* from live query statistics (`pg_stat_statements`); a manipulation check verifies the derivation is exact on every seed. Second, the replica lag is emergent (network-emulated, not dialed). Graded, the naive monitor is 0.46–0.47 and the WAL-witnessed certificate over the derived footprint 0.01–0.03. The guard rule that makes the certificate robust—a witness horizon matched to the truth oracle’s guard—generalizes directly to the geo setting: each replica’s own minimum round-trip time sets its guard.

C. Geo-distributed nearest-certified routing

The geo cell is the read-routing endgame and the bridge to the routing instance of the program. A PostgreSQL primary and three streaming replicas are placed in three real, geographically separated datacenters on the National Research Platform. Three routers compete on two axes: *nearest* minimizes round-trip time, *least-lag* minimizes replica lag, and *certified* routes to the nearest replica whose footprint certificate clears against its own witnessed replay LSN. Graded, the certified router attains a false-clear of 0 at round-trip ≈ 11 ms, while nearest pays a false-clear of ≈ 0.30 (it routes local but stale) and least-lag pays 16–18 ms (it routes fresh but far). The certificate is thus *fresher than nearest and more local than least-lag*: it is the only router that reads both the consumer’s footprint and the witness. A guard-consistency lesson recurs from the production cell—the witness horizon must match the truth oracle’s guard, or every replica is disqualified by in-flight writes and the router falls back to the distant primary.

V. THE CONSISTENCY DESIGN SPACE

A survey of the replication and consistency design space through the certificate lens (archived with the program) organizes the space by one claim: the *form* of the certificate follows the order structure of the consistency model. A total order (consensus, strongly consistent stores) admits a scalar log-position certificate—the LSN or oplog timestamp used above. A partial order (leaderless, causal, CRDT stores [6]) requires a version-set dominance certificate. A probabilistic model [3] requires a calibrated depth. A physical-time model [4] requires a lease. The witness leg is near-universal, and prior art already ships pieces: Nakamoto confirmations and Spanner commit-wait are *already* calibrated; session guarantees [5] and vendor bounded-staleness reads are consumer-relative by construction but *uncalibrated*. The program’s delta is to ship all four grammar words together—consumer-relative, witnessed, calibrated, derivable—and to *measure the rate*. ZooKeeper’s local-read staleness under `sync` [7], [8] is a natural next cell.

VI. DISCUSSION

CAP and PACELC. The footprint certificate is the constructive form of the staleness face of consistency. Where CAP [9] and PACELC [10] name a tradeoff between consistency and latency (or availability), the certificate *quantifies the interior*: it prices the consistency each consumer actually needs, per read, and routes to the nearest replica that supplies it. Partition is the limiting case where the witnessed staleness diverges; the else-latency regime is exactly where nearest-certified routing wins. *Age of information*. As in the radio-access instance [11], age bounds staleness on the log; the footprint certificate measures its consequence for the specific consumer, which is what a consistency SLA must bound. *Benchmarking*. The cells are amenable to YCSB-lineage workloads [12]; the footprint is exactly the per-record read distribution such benchmarks parameterize.

VII. LIMITATIONS AND EXTERNAL VALIDITY

The lab and production cells are single-primary, single- or few-replica; the geo cell is three replicas across three datacenters but still PostgreSQL streaming replication. Footprints are diagonal read-operators (per-object weights); a full P_C over query-result functionals is a natural extension already used in the compression instance [2]. Induced staleness in the geo cell is partly disclosed (a rotating apply-pause) because the research backbone is faster than the guard; the locality axis is real inter-datacenter round-trip. The consistency-model taxonomy of Section V is argued from the survey, not measured cell-by-cell; leaderless and CRDT certificates are future cells.

VIII. CONCLUSION

Replication freshness is not a scalar property of a replica but a relation between its drift and what a consumer reads. A replica-lag threshold is two-sidedly vacuous on a heterogeneous workload; a footprint certificate $\text{tr}(P_C \Sigma)$, graded against the replication log the substrate already maintains, holds at a percent-level false-clear across PostgreSQL, MongoDB, a production instance with derived footprints, and a three-datacenter fleet. Nearest-certified read routing is simultaneously fresher than nearest and more local than least-lag. We recommend that bounded-staleness reads be specified, measured, and *certified* against the consumer’s footprint and the replication witness, rather than a global lag bound.

REPRODUCIBILITY

Each cell’s pre-registration, coded verdict runner (cooling-off and substrate guards), family record, and graded verdict are archived with the program [13]; substrates are unmodified PostgreSQL and MongoDB.

REFERENCES

- [1] A. H. Bond, “Observation theory: Geometry, distortion, and reliability as properties of observation,” *Submitted to IEEE Transactions on Information Theory*, 2026.
- [2] —, “TurboQuant: Consumer-relative compression with witnessed rank certificates,” <https://github.com/ahb-sjsu/turboquant-pro>, 2026.
- [3] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, and I. Stoica, “Probabilistically bounded staleness for practical partial quorums,” in *Proc. VLDB Endowment*, vol. 5, no. 8, 2012.
- [4] J. C. Corbett *et al.*, “Spanner: Google’s globally-distributed database,” in *USENIX OSDI*, 2012.
- [5] D. B. Terry, A. J. Demers, K. Petersen, M. J. Spreitzer, M. M. Theimer, and B. B. Welch, “Session guarantees for weakly consistent replicated data,” in *Proc. Int. Conf. Parallel and Distributed Information Systems (PDIS)*, 1994.
- [6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Symp. Self-Stabilizing Systems (SSS)*, 2011.
- [7] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for internet-scale systems,” in *USENIX Annual Technical Conference*, 2010.
- [8] F. P. Junqueira, B. C. Reed, and M. Serafini, “Zab: High-performance broadcast for primary-backup systems,” in *IEEE/IFIP DSN*, 2011.
- [9] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, 2002.
- [10] D. J. Abadi, “Consistency tradeoffs in modern distributed database system design,” *IEEE Computer*, vol. 45, no. 2, 2012.
- [11] S. Kaul, R. Yates, and M. Gruteser, “Real-time status: How often should one update?” in *IEEE INFOCOM*, 2012.
- [12] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *ACM Symp. Cloud Computing (SoCC)*, 2010.
- [13] A. H. Bond, “Sealed pre-registrations and claim ledger for the observation-theory program,” <https://github.com/ahb-sjsu/network-governor>, 2026.